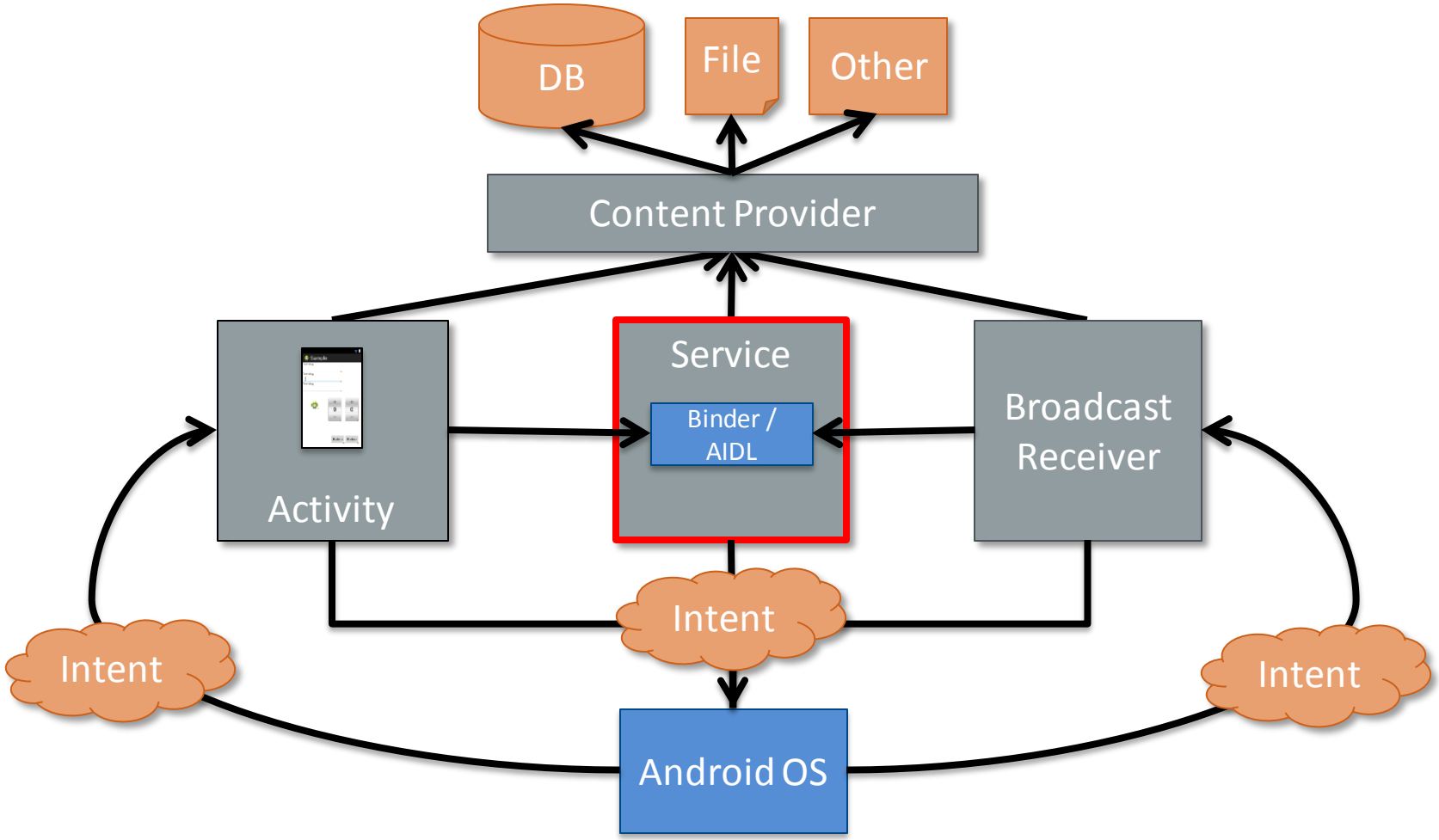
605.486 – Android Development

Module 16
Services

Application Components

and Typical Communication



Background Threads

- Only within an application
- Usually just for one activity
- Raw Threads
- AsyncTask

Services

- Across applications
- Share processing
- Share data
- Provide access to a resource/sensor/external

Service Types

- STARTED
  - Launched to perform a task
  - Keeps running in background
  - Components can connect & communicate with it
  - Stays alive even when nothing is connected to it
    - Other components can stop()
    - Service can call stopSelf()
    - Unless process is killed by system!
    - Less likely if started as “foreground” service, like music player
- BOUND
  - Higher priority (less likely to be killed)
  - Connected to components
  - “Binding” provides an API with which to communicate
  - Stays alive as long as at least one connection remains

Where is a service useful? (1)

- Keeping running when app is not being used by user
  - Downloader
  - Music player
- Providing API to access common resources/data

Where is a service useful? (2)

- Watching and notifying
  - Can perform sendBroadcast(intent) to communicate with interested activities that set up broadcast receivers
  - If bound, can use registered callbacks
- Responding to broadcasts (NFC tag read, for example)
  - Manifest-registered Broadcast receiver catches intent
    - Short processing: handle in receiver itself (only if very quick!)
    - Long processing: start a service (frees up system thread)
    - Rarely start activity

Example 1 - Sample Service w/ log

- onCreate()
  - Service instance has been created
- onDestroy()
  - Service instance is being destroyed by android
- onStartCommand(intent, flags, id)
  - Processing when startService(intent) has been called
  - Unique id passed every time - returned
- onBind(intent)
  - Called to create communication binder

Example 2 - Using a thread

- Create thread in onStartCommand
- Be sure to call stopSelf() at end of processing
- Note that onDestroy called, but thread not killed!

Example 3 - Stopping the thread

- Keep track of thread in service
- Call interrupt() on the thread from onDestroy()
- Add !isInterrupted() to the loop
- Logging is *very* helpful here to ensure thread is running/stopping

Example 4 - Bound Service (1)

- Define API class (typically inner class)
  - Extends Binder
  - Defines method to perform work
    - Example: resetCounter()
- Create instance
- Override onBind(intent)
  - Start processing (if needed)
  - Return binder instance
- NOTE: This binder only works in same process!

Example 4 - Bound Service (2)

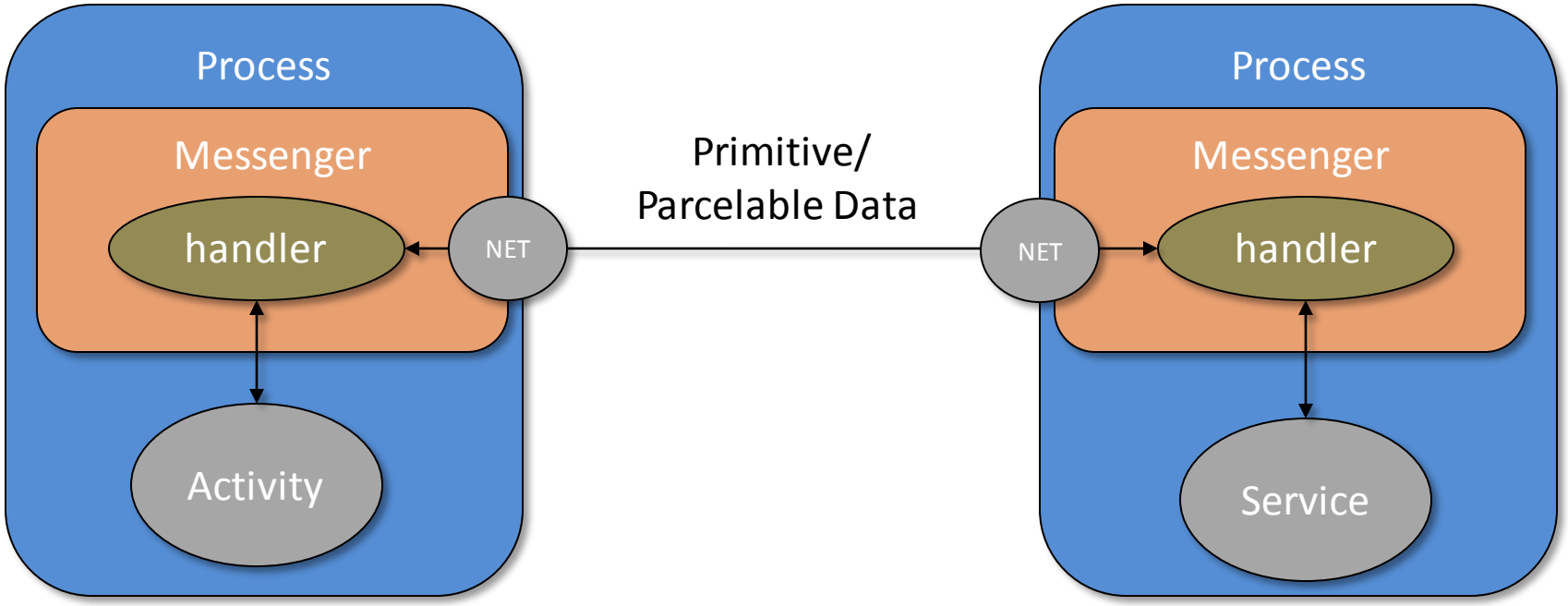
- Caller creates implementation of ServiceConnection
  - onServiceConnected(name, binder)
    - This hands the binder (or proxy) to the caller – hold onto it
  - onServiceDisconnected(name)
    - Stop using the binder when called
- Call bindService to connect
  - Pass ServiceConnection instance – called when started
- Call unbindService to disconnect
  - Pass ServiceConnection instance – called when disconnected
- Call your API methods in the binder normally

Example 5 - Service feedback

- Add callback for feedback to caller
  - Service defines interface
  - Add registration to API
- Adding second activity to show multiple bindings
- Bind/unbind in activity onStart/onStop
- Progress bar
  - requestWindowFeature(Window.*FEATURE_PROGRESS);*
  - *setProgress(int)*
- Note overlap of onStart/onStop in activities
  - Good because it retains binding!

Example 6 - Remote service

- Sharing service across processes



- If no callback – only need handler on service side
- Callback requires handlers on both ends

Example 7 - Remote service callback

- Add handler to activity
- Send message to service to register
  - Pass activity's messenger as "replyTo" on a message

Example 8 - AIDL

- Messenger/handler works - But it's gross!
- Android Interface Definition Language (AIDL)
  - Describes inter-process communication
  - Generates proxies
- RemoteService2.aidl
  - Describes method calls to service
  - NOTE: ***Imports required even for same package!***
- RemoteService2Reporter.aidl
  - Note "in" for parameter direction – optimization
- RemoteService2Impl